

Лекція 13

Основи TypeScript.



TypeScript

- **TypeScript** — це статично типізована та компільована мова на основі JavaScript. **Superset of Javascript** - додає типізацію та робить код більш передбачуваним і безпечним.
- Код, написаний на TypeScript, компілюється у JavaScript
- Сумісний з JavaScript. Підтримка стандартів JS
- Виявлення помилок на етапі компіляції
- Реалізує концепції, які властиві об'єктно-орієнтованим мовам, як, наприклад, наслідування, поліморфізм, інкапсуляція та модифікатори доступу тощо
- TypeScript дозволяє швидше та простіше писати великі складні комплексні програми, відповідно їх легше підтримувати, розвивати, масштабувати та тестувати.
- Open source on GitHub <https://github.com/Microsoft/TypeScript>

Getting started

- Install NodeJs
- `npm install -g typescript`
- `tsc myFile.ts`

Параметри компілятора

- Автоматична перекомпіляція
tsc -w app.ts
- Версія ECMAScript
tsc app.ts -t ES5
- Встановлення каталогу
tsc --outDir result app.ts
- Тип модуля
tsc -m commonjs app.ts
- Справка
tsc -h

<https://www.typescriptlang.org/docs/handbook/compiler-options.html>

tsconfig.json

Генеруємо tsconfig.json

npx tsc -init

```
tsconfig.json ×
1 {
2   // Visit https://aka.ms/tsconfig to read more about this file
3   "compilerOptions": {
4     // File Layout
5     // "rootDir": "./src",
6     // "outDir": "./dist",
7
8     // Environment Settings
9     // See also https://aka.ms/tsconfig/module
10    "module": "nodenext",
11    "target": "esnext",
12    "types": [],
13    // For nodejs:
14    // "lib": ["esnext"],
15    // "types": ["node"],
16    // and npm install -D @types/node
17
18    // Other Outputs
19    "sourceMap": true,
20    "declaration": true,
21    "declarationMap": true,
22
23    // Stricter Typechecking Options
24    "noUncheckedIndexedAccess": true,
25    "exactOptionalPropertyTypes": true,
26
27    // Style Options
28    // "noImplicitReturns": true,
29    // "noImplicitOverride": true,
30    // "noUnusedLocals": true,
31    // "noUnusedParameters": true,
32    // "noFallthroughCasesInSwitch": true,
33    // "noPropertyAccessFromIndexSignature": true,
34
35    // Recommended Options
36    "strict": true,
```

compilerOptions

- **baseUrl**: Вказує базовий шлях для модулів.
- **target**: Вказує, в яку версію ECMAScript повинен компілювати код.
- **module**: Вказує, який формат модулів повинен використовувати TypeScript під час створення коду.
- **outDir**: Вказує, в яку директорію складати скомпільовані файли.
- **noEmitOnError**: Забороняє компіляцію файлів TypeScript, якщо під час компіляції виникають помилки.
- **esModuleInterop**: Вказує, що TypeScript повинен генерувати модулі у форматі ES-модулів, щоб забезпечити сумісність із CommonJS-модулями, які використовуються у середовищах Node.js.
- **strict**: Включає всі строгі налаштування компілятора. Це включає строгу перевірку типів, перевірку присвоювань та інші строгі правила компіляції.
- **allowJs**: Вказує, чи можна компілювати файли JavaScript разом із файлами TypeScript.
- **resolveJsonModule**: Дозволяє імпортувати модулі JSON безпосередньо у TypeScript.

tsconfig.json

- **exclude**: Визначає, які файли чи директорії мають бути виключені з компіляції TypeScript.
- **include**: Визначає, які файли чи директорії мають бути включені у процес компіляції TypeScript. Тут вказуються файли або директорії, які мають бути скомпільовані у JavaScript.

```
"exclude": [  
  "node_modules",  
  "dist"  
],  
  
"include": [  
  "src/**/*.ts"  
]
```

Файл tsconfig.json використовується при компіляції, якщо компілятору не передаються назви файлів, які потрібно скомпілювати, він переглядає поточний каталог, шукає в ньому файл tsconfig.json і потім при компіляції використовує ті параметри, які визначені в цьому файлі.

Якщо компілятору передаються назви файлів, наприклад, **tsc app.ts**, то файл tsconfig.json ігнорується.

Тобто, якщо в вас є tsconfig.json, то для компіляції можна просто виконати команду **tsc**

Повний список опцій в tsconfig.json:

<https://www.typescriptlang.org/tsconfig>

Базові типи даних

- **boolean**: логічне значення true чи false
- **number**: числове значення
- **string**: строки
- **Array**: масиви
- **кортежі**
- **enum**: перерахування
- **any**: довільний тип
- **Symbol**
- **null i undefined**: відповідають значенням null i undefined у javascript
- **never**: також представляє відсутність значення і використовується як тип функцій, що повертаються, які генерують або повертають помилку

Базові типи даних

boolean

```
let isEnabled: boolean = true;  
let isAlive: boolean = false;
```

number

```
let age: number = 36;  
let height: number = 1.68;  
  
let decimal: number = 6;  
// шістнадцяткова система  
let hex: number = 0xf00d;           // 61453 у десятичній  
// двійкова система  
let binary: number = 0b1010;       // 10 у десятичній  
// вісімкова система  
let octal: number = 0o744;         // 484 у десятичній
```

bigint дозволяє зберігати числа більше, ніж $2^{53} - 1$

```
const num1: bigint = BigInt(100);  
const num2: bigint = 100n;
```

цей тип – частина стандарту ES2020

```
"compilerOptions": {  
  "target": "es2020"  
}
```

Базові типи даних

string

```
const productName: string = 'iPhone';
const productModel: string = "13";
const price: number = 999;
const taxRate: number = 0.2;
const getTotalPrice = (price: number, taxRate: number): number => {
  return price + (price * taxRate);
};
const info: string = `
  Product: ${productName} ${productModel}
  Price: ${price}
  Tax (20%): ${price * taxRate}
  Total Price: ${getTotalPrice(price, taxRate)}
`;
/**
 * Product: iPhone 13
 * Price: $999
 * Tax (20%): $199.8
 * Total Price: $1198.8
 */
console.log(info);
```

Базові типи даних

Array

```
const list: number[] = [10, 20, 30];  
const colors: string[] = ["red", "green", "blue"];  
  
const names: Array<string> = ["Tom", "Bob", "Alice"];  
console.log(names[1]); // Bob
```

ReadonlyArray

```
const people: ReadonlyArray<string> = ["Tom", "Bob", "Sam"];  
const people: readonly string[] = ["Tom", "Bob", "Sam"];  
  
people[1] = "Kate"; // ! Index signature in type readonly string[] only  
permits reading  
people.push("Kate"); // ! Property push does not exist on type readonly  
string[]  
people.pop(); // ! Property pop does not exist on type readonly string[]  
  
const people: ReadonlyArray<string> = new ReadonlyArray("Tom", "Bob",  
"Sam"); // ! ReadonlyArray only refers to a type, but is being used as a  
value here
```

Базові типи даних

Кортежі

```
const user: [string, number];  
user = ["Tom", 28];  
user = [28, "Tom"]; // ! Помилка
```

```
console.log(user[1]); // 28  
user[1] = 37;  
console.log(user[1]); // 37
```

```
const bob: [string, number, boolean?] = ["Bob", 41, true];  
const tom: [string, number, boolean?] = ["Tom", 36];
```

```
let math: [string, ...number[]] = ["Math", 5, 4, 5, 4, 4];  
let physics: [...number[], string] = [5, 5, 5, "Physics"];  
let chemistry: [string, ...number[], boolean] =  
["Chemistry", 3, 3, 4, 5, false];
```

```
const tom: readonly [string, number] = ["Tom", 36];  
tom[1] = 37; // ! Помилка
```

Базові типи даних

Тип **enum** або перерахування дозволяє визначити набір іменованих констант, які описують певні стани

```
enum Season { Winter, Spring, Summer, Autumn };
```

```
let current: Season = Season.Summer;  
console.log(current);           // 2  
current = Season.Autumn;       // зміна значення  
console.log(current);           // 3
```

```
enum Season { Winter = 4, Spring = 8, Summer = 16, Autumn = 32 };
```

```
var current: string = Season[16]; // 16 - числове значення Summer  
console.log(current);             // Summer
```

```
enum Season { Winter = 1, Spring = "Весна", Summer = 3, Autumn = "Осінь" };  
console.log(Season.Summer);       // 3  
console.log(Season.Autumn);       // Осінь
```

Базові типи даних

never

```
function throwError(message: string): never {  
    throw new Error(message)  
}
```

any

```
let someVar: any = 'hello';  
console.log(someVar);    // зараз someVar - це string  
someVar = 20;  
console.log(someVar);    // зараз someVar - це number  
const someArray: any[] = [24, "Tom", false];
```

typeof

```
let sum: any;  
sum = 1200;  
if (typeof sum === "number") {  
    let result: number = sum / 12;  
    console.log(result);  
} else {  
    console.log("invalid operation");  
}
```

Базові типи даних

typeof

```
let num: number = 42;
let str: string = "Hello";
let bool: boolean = true;
let obj: object = {};
let func: Function = () => {};
let undef: undefined;
let sym: symbol = Symbol();
let bigint: bigint = BigInt(10);

console.log(typeof num);    // "number"
console.log(typeof str);    // "string"
console.log(typeof bool);   // "boolean"
console.log(typeof obj);    // "object"
console.log(typeof func);   // "function"
console.log(typeof undef);  // "undefined"
console.log(typeof sym);    // "symbol"
console.log(typeof bigint); // "bigint"
```

Базові типи даних

Symbol

```
const firstName = Symbol('This is the first name');  
const lastName = Symbol('This is the last name');  
const address = 'ADDRESS';
```

// Створюємо об'єкт з використанням символів в якості полів

```
let person = {  
  [firstName]: 'John',  
  [lastName]: 'Doe',  
  [address]: '123 Easy St, Procrastination Town, LA 247',  
  age: 30,  
};
```

// Доступ до значень через символи

```
console.log(person[firstName]); // 'John'  
console.log(person[lastName]); // 'Doe'  
console.log(person[address]); // '123 Easy St, Procrastination Town, LA 247'
```

// Символи не перераховуються при переборі полів об'єкта

```
for (let key in person) {  
  console.log(key); // 'ADDRESS', 'age' - символи firstName та lastName не виводяться  
}
```

// Отримуємо масив символів об'єкта

```
const symbols = Object.getOwnPropertySymbols(person);  
console.log(symbols); // [ Symbol(This is the first name), Symbol(This is the last name) ]
```



```
const add = (a: number, b: number): number => a + b;  
console.log(add(2, 2));
```

```
const add = (a: number, b: number): void => {  
  console.log(a + b);  
};  
add(10, 20);
```

```
const getName = (  
  firstName: string,  
  lastName?: string  
) : string => lastName ? `${firstName} ${lastName}` : firstName;  
let name1 = getName("Christopher", "Davis");  
console.log(name1); // Christopher Davis  
let name2 = getName("Michael");  
console.log(name2); // Michael
```

```
const getName = (  
  firstName: string,  
  lastName: string = "Smith"  
) => `${firstName} ${lastName}`;
```

Присвоювання функцій

Передача функцій в якості параметра

```
const hello = (name: string) => {  
  console.log(`Hello ${name}`);  
};  
let message: (name: string) => void;  
message = hello;  
message('TypeScript'); // Hello TypeScript
```

```
type Operation = (a: number, b: number) => number;
```

```
const mathOp = (x: number, y: number, op: Operation): number => op(x, y);
```

```
const sum: Operation = (x: number, y: number): number => x + y;
```

```
console.log(mathOp(10, 20, sum)); // 30
```

Об'єднання Union

```
const printId = (id: number | string) => {  
  console.log(`Id: ${id}`);  
}
```

```
let id: string | number = "ruy74";  
printId(id); // Id: ruy74  
id = 9876;  
printId(id); // Id: 9876
```

```
const printSentence = (words: string[] | string) => {  
  if (typeof words === "string") {  
    console.log(words);  
  } else {  
    console.log(words.join(" "));  
  }  
}  
printSentence(["Мова", "програмування", "TypeScript"]);  
printSentence("Мова програмування JavaScript");
```

Псевдоніми типів

```
type Id = number | string;
```

```
const printId = (id: Id) => {  
  console.log(`Id: ${id}`);  
}
```

```
let id: Id = "ruy74";  
printId(id); // Id: ruy74  
id = 9876;  
printId(id); // Id: 9876
```

```
type Person = { name: string; age: number };
```

```
let tom: Person = { name: "Tom", age: 36 };  
let bob: Person = { name: "Bob", age: 41 };
```

```
function printPerson(user: Person) {  
  console.log(`Name: ${user.name} Age: ${user.age}`);  
}  
printPerson(tom);  
printPerson(bob);
```

Розширення псевдонімів

```
type Person = { name: string; age: number };  
// розширюємо псевдонім Person  
// type Employee = { name: string; age: number; company: string };  
type Employee = Person & { company: string };  
  
let tom: Person = { name: "Tom", age: 36 };  
let bob: Employee = { name: "Bob", age: 41, company: "Microsoft" };  
  
const printPerson = (user: Person) => {  
  console.log(`Name: ${user.name} Age: ${user.age}`);  
}  
  
printPerson(tom);  
printPerson(bob);
```

Класи. Поля класів

```
class User {  
    name: string;  
    age: number;  
}
```

```
const tom: User = new User();
```

```
tom.name = "Tom";
```

```
tom.age = 35;
```

```
console.log(`name: ${tom.name}   age: ${tom.age}`);
```

```
// name: Tom   age: 35
```

Методи. Конструктори

```
class User {  
    name: string;  
    age: number;  
  
    constructor(userName: string, userAge: number) {  
        this.name = userName;  
        this.age = userAge;  
    }  
  
    updateUserInfo(newName: string, newAge: number): void {  
        this.name = newName;  
        this.age = newAge;  
    }  
  
    toString(): string {  
        return `${this.name}: ${this.age}`;  
    }  
}
```

Поля readonly

```
class User {  
  
    readonly name: string = "Default user";  
    age: number;  
  
    constructor(name: string, age: number) {  
        this.name = name;  
        this.age = age;  
    }  
  
    print(){  
        console.log(`name: ${this.name}   age: ${this.age}`);  
    }  
}  
  
let tom = new User("Tom", 36);  
tom.name = "Bob";           // ! Cannot assign to name because it is a  
                             read-only property  
tom.print();                 // name: Tom   age: 36
```


Наслідування

```
class Person {  
    name: string;  
    constructor(name: string) {  
        this.name = name;  
    }  
  
    print(): void {  
        console.log(`Name: ${this.name}`);  
    }  
}  
  
class Employee extends Person {  
    company: string;  
  
    work(): void {  
        console.log(`${this.name} works at the company ${this.company}`);  
    }  
}  
  
let bob: Employee = new Employee("Bob");  
bob.print();  
bob.company = "Microsoft";  
bob.work();
```

Перевизначення конструктора

```
class Person {  
  name: string;  
  constructor(name: string) {  
    this.name = name;  
  }  
  print(): void {  
    console.log(`Name: ${this.name}`);  
  }  
}
```

```
class Employee extends Person {  
  company: string;  
  constructor(name: string, company: string) {  
    super(name);  
    this.company = company;  
  }  
  work(): void {  
    console.log(`${this.name} works at the company ${this.company}`);  
  }  
}
```

```
let bob: Employee = new Employee('Bob', 'Microsoft');
```

Перевизначення методів

```
class Person {  
  name: string;  
  constructor(name: string) {  
    this.name = name;  
  }  
  print(): void {  
    console.log(`Name: ${this.name}`);  
  }  
}
```

```
class Employee extends Person {  
  company: string;  
  constructor(name: string, company: string) {  
    super(name);  
    this.company = company;  
  }  
  print(): void {  
    super.print();  
    console.log(`Works at the company: ${this.company}`);  
  }  
}
```

```
let bob: Employee = new Employee("Bob", "Microsoft");  
bob.print();
```

Абстрактні класи

```
abstract class Figure {  
  abstract getArea(): number;  
}
```

```
class Rectangle extends Figure {  
  width: number;  
  height: number;
```

```
  constructor(width: number, height: number){  
    super();  
    this.width = width;  
    this.height = height;  
  }
```

```
  getArea(): number {  
    return this.width * this.height;  
  }  
}
```

```
let someFigure: Figure = new Rectangle(20, 30)  
console.log(someFigure.getArea());
```

Модифікатори доступу

```
class MyFileSystem {  
    private files: string[] = [];  
  
    public addFile(fileName: string): void {  
        this.files.push(fileName);  
        console.log(`File added: ${fileName}`);  
    }  
  
    protected getFileList(): string[] {  
        return this.files;  
    }  
  
    protected clearFiles(): void {  
        this.files = [];  
        console.log("File list cleared.");  
    }  
}
```

```
let fileManager = new FileManager();  
fileManager.addFile("Document1.txt");  
fileManager.addFile("Image.jpg");  
fileManager.listFiles();  
fileManager.deleteAllFiles();  
fileManager.listFiles();
```

```
class FileManager extends MyFileSystem {  
    public listFiles(): void {  
        const fileList = this.getFileList();  
        console.log("File list:");  
        fileList.forEach((file, index) => {  
            console.log(`${index + 1}. ${file}`);  
        });  
    }  
}
```

```
    public deleteAllFiles(): void {  
        this.clearFiles();  
        console.log("All files deleted.");  
    }  
}
```

Модифікатори доступу

```
class Person {  
  
    constructor(private name: string, private age: number) { }  
  
    printPerson(): void {  
        console.log(`Name: ${this.name}   Age: ${this.age}`);  
    }  
}
```

```
class Person {  
    constructor(private readonly name: string, private age: number) { }  
}
```

Статичні поля та методи

```
class Person {  
  
    private age: number;  
    private name: string;  
  
    private static readonly retirementAge: number = 65;  
  
    public static calculateYears(age: number): number {  
        return Person.retirementAge - age;  
    }  
  
    constructor(name: string, age: number) {  
        this.name = name;  
        this.age = age;  
    }  
}  
  
let tom = new Person("Tom", 36);  
let years = Person.calculateYears(36);  
console.log(`До пенсії залишилось: ${years} років`);
```

Интерфейсы

```
interface IUser {  
  readonly id: number;  
  name: string;  
  age?: number;  
  
  sayWords(words: string): void;  
}  
  
let employee: IUser = {  
  id: 1,  
  name: "Tom",  
  
  sayWords: function(words: string): void {  
    console.log(`${this.name} говорить "${words}"`);  
  }  
}
```


Интерфейси

```
interface IUser {  
    id: number;  
    name: string;  
    getFullName(surname: string): string;  
}
```

```
class User implements IUser {  
    id: number;  
    name: string;  
    age: number;  
    constructor(userId: number, userName: string, userAge: number) {  
        this.id = userId;  
        this.name = userName;  
        this.age = userAge;  
    }  
    getFullName(surname: string): string {  
        return this.name + " " + surname;  
    }  
}
```

```
let tom: IUser = new User(1, "Tom", 23);  
console.log(tom.getFullName("Simpson"));
```

Розширення інтерфейсів

```
interface IUser {  
  id: number;  
  name: string;  
}
```

```
interface IUser{  
  age: number;  
}
```

```
const employee: IUser = {  
  id: 1,  
  name: "Alice",  
  age: 31  
}
```

Наслідування інтерфейсів

```
interface IMovable {  
    speed: number;  
    move(): void;  
}
```

```
interface ICar extends IMovable {  
    fill(): void;  
}
```

```
class Car implements ICar {  
    speed: number;  
    move(): void {  
        console.log(  
            `The car is moving at a speed of ${this.speed} km/h`  
        );  
    }  
    fill(): void {  
        console.log("Filling the car...");  
    }  
}
```

```
let auto = new Car();  
auto.speed = 60;  
auto.fill();  
auto.move();
```

```
interface Person {  
  name: string;  
  age: number;  
  email: string;  
};
```

```
type PersonKeys = keyof Person; // 'name' | 'age' | 'email'
```

```
const getValueByKey = (  
  key: PersonKeys,  
  person: Person  
) : Person[PersonKeys] => {  
  return person[key];  
}
```

```
const person: Person = {  
  name: 'Alice',  
  age: 30,  
  email: 'alice@example.com',  
};
```

```
console.log(getValueByKey('name', person)); // Alice  
console.log(getValueByKey('age', person)); // 30  
console.log(getValueByKey('email', person)); // alice@example.com
```

Перетворення типів

```
class Person {  
    name: string;  
    constructor(userName: string) {  
        this.name = userName;  
    }  
}
```

```
class Employee extends Person {  
    company: string;  
    constructor(userName: string, company: string) {  
        super(userName);  
        this.company = company;  
    }  
}
```

```
const tom: Person = new Employee("Tom", "Microsoft");  
console.log(tom.company); // ! Property company does not exist on type Person
```

```
console.log((<Employee>tom).company);
```

```
console.log((tom as Employee).company);
```

instanceof

```
const tom = new Employee('Tom', 'Microsoft');
```

```
if (tom instanceof Person) {  
  console.log('Tom is a Person');  
} else {  
  console.log('Tom is not a Person');  
}
```

instanceof

```
interface Shape {  
  calculateArea(): number;  
}
```

```
class Circle implements Shape {  
  constructor(private radius: number) {}  
  calculateArea() {  
    return Math.PI * Math.pow(this.radius, 2);  
  }  
}
```

```
const printArea = (shape: Shape) => {  
  if (shape instanceof Square) {  
    const area = (shape as Square).doubleArea();  
    console.log(`Double area of the square: ${area}`);  
  } else {  
    const area = shape.calculateArea();  
    console.log(`Figure area: ${area}`);  
  }  
}
```

```
class Square implements Shape {  
  constructor(private sideLength: number) {}  
  calculateArea() {  
    return Math.pow(this.sideLength, 2);  
  }  
  doubleArea() {  
    return this.calculateArea() * 2;  
  }  
}
```

```
const circle = new Circle(5);  
const square = new Square(4);  
printArea(circle); // Figure area: 78.54  
printArea(square); // Double area of the square: 32
```

Generics

```
const getId = <T>(id: T): T => {  
  return id;  
}
```

```
const result1 = getId<number>(5);  
const result2 = getId<string>('abc');
```

```
const getString = <T>(args: Array<T>): string => {  
  return args.join(", ");  
}
```

```
const result = getString<number>([1, 2, 34, 5]);  
const result2 = getString<string>(['one', 'two', 'three']);
```


Generics

```
class User<T> {  
    private id: T;  
  
    constructor(id: T) {  
        this.id = id;  
    }  
  
    getId(): T {  
        return this.id;  
    }  
}
```

```
class ObjectId {  
    // ... some implementation  
}
```

```
let tom = new User<number>(3);  
let alice = new User<string>('abc123');  
let bob = new User<ObjectId>(new ObjectId());
```

Generics

```
class User {  
    private name: string;  
  
    constructor(name: string) {  
        this.name = name;  
    }  
    getName(): string {  
        return this.name;  
    }  
}
```

```
const compareName = <T extends User>(obj1: T, obj2: T): void => {  
    obj1.getName() === obj2.getName()  
        ? console.log('Names are the same')  
        : console.log('Names are not the same')  
};
```

```
const bob = new User('Bob');  
const bobic = new User('Bob');  
compareName<User>(bob, bobic);
```

Generics

```
class User {  
  private name: string;  
  constructor(name: string) {  
    this.name = name;  
  }  
  getName(): string {  
    return this.name;  
  }  
}
```

```
interface NameProvider {  
  getName: Function,  
}
```

```
const compareName = <T extends NameProvider>(obj1: T, obj2: T): void => {  
  obj1.getName() === obj2.getName()  
    ? console.log('Names are the same')  
    : console.log('Names are not the same');  
};
```

```
const bob = new User('Bob');  
const bobic = {  
  getName(): string {  
    return 'Bob'  
  }  
};  
compareName(bob, bobic);
```

Utility types

TypeScript provides several utility types to facilitate common type transformations. **These utilities are available globally.**

- **Record**
- **Readonly**
- **Partial**
- **Pick**
- **Omit**
- **Required**
- **Exclude**
- **Extract**

Record

```
type NumberRecord = Record<string, number[]>;
```

```
const data: NumberRecord = {  
  'one': [1, 2, 3],  
  'two': [4, 5],  
  'three': [6, 7, 8, 9],  
};
```

```
console.log(data['one']); // [1, 2, 3]  
console.log(data['two']); // [4, 5]
```

// Реалізація

```
type Record<K extends keyof any, T> = {  
  [P in K]: T;  
};
```

Readonly

```
interface IAnimalEntity {  
  name: string;  
  age: number;  
}
```

```
const json = `{  
  "name": "animal",  
  "age": 0  
}`;
```

```
const animal: Readonly<IAnimalEntity> = JSON.parse(json);
```

`animal.name = 'newanimal';` // Cannot assign to name because it is a read-only property.

`animal.age = 0;` // ! Cannot assign to age because it is a read-only property.

Partial

```
interface User {  
  id: number;  
  name: string;  
  age: number;  
  email: string;  
}
```

```
const updateUser = (user: User, updates: Partial<User>): User => {  
  return { ...user, ...updates };  
}
```

```
const user: User = {  
  id: 1,  
  name: 'Alice',  
  age: 30,  
  email: 'alice@example.com',  
};
```

```
const updatedUser = updateUser(user, { name: 'Alice Smith', age: 31 });
```

```
console.log(updatedUser);
```

```
interface Article {  
  title: string;  
  content: string;  
  author: string;  
  publicationDate: Date;  
  tags: string[];  
  isPublished: boolean;  
}
```

```
type ArticlePreview = Pick<Article, 'title' | 'author' | 'publicationDate'>;
```

```
const displayArticlePreview = (preview: ArticlePreview) => {  
  console.log(`Title: ${preview.title}`);  
  console.log(`Author: ${preview.author}`);  
  console.log(`Publication Date: ${preview.publicationDate.toString()}`);  
}
```



```
interface BookingDetails {  
    flightNumber: string;  
    departureDate: Date;  
    arrivalDate: Date;  
    departureAirport: string;  
    arrivalAirport: string;  
    passengerName: string;  
    passengerEmail: string;  
    paymentDetails: {  
        cardNumber: string;  
        expiryDate: string;  
        cvv: string;  
    };  
}
```

```
type BookingDataForStorage = Omit<BookingDetails, 'paymentDetails'>;
```

```
const sendBookingInfo = (bookingData: BookingDataForStorage) => {  
    /* ... */  
}
```

Required

```
interface User {  
  id: string  
  email?: string  
  name?: string  
  age?: number  
}
```

```
function saveToDb(user: Required<User>) {  
  user.email.toLowerCase();  
  user.age.toFixed(0);  
}
```

```
const draft: User = {  
  id: '1',  
  email: 'test@mail.com',  
}
```

```
saveToDb(draft); // Error!
```

```
type RequiredEmail = Required<Pick<User, 'email'>>
```

```
function sendEmail(user: RequiredEmail) {  
  // ...  
}
```

Exclude

```
type Color = 'red' | 'green' | 'blue' | 'yellow' | 'purple';
```

```
type NonYellowPurpleColors = Exclude<Color, 'yellow' | 'purple'>;
```

```
const color: NonYellowPurpleColors = 'red';
```

```
const color2: NonYellowPurpleColors = 'yellow'; // ! Type "yellow" is  
not assignable to type NonYellowPurpleColors
```

```
const color3: NonYellowPurpleColors = 'purple'; // ! Type "purple" is  
not assignable to type NonYellowPurpleColors
```

```
const color4: NonYellowPurpleColors = 'green';
```

```
class ObjectId {  
  // ... some implementation  
}
```

```
type UserId = string | number | ObjectId;
```

```
type UserNumberId = Exclude<UserId, string | ObjectId>
```

Extract

```
type Status = 'draft' | 'published' | 'archived'
```

```
type PublicStatus = Extract<Status, 'published' | 'archived'>  
// 'published' | 'archived'
```

```
interface Person {  
  name: string;  
  age: number;  
  job: string;  
}
```

```
interface Animal {  
  name: string;  
  age: number;  
  species: string;  
}
```

```
type Intersection = Extract<keyof Person, keyof Animal>; // "name" | "age"
```

Decorators

Decorators are special **functions**, marked with the **@** symbol, that allow you to annotate and modify

- **classes**
- **methods**
- **properties**
- **parameters**

at design time, adding metadata or changing behavior without altering the original code structure, making them powerful for frameworks (like Angular, NestJs) and meta-programming tasks such as

- logging
- validation
- time measurement
- authorization
- dependency injection

Decorators are a way to hang behavior next to your code, rather than inside it.

```
tsconfig.json: {  
  "compilerOptions": {  
    "experimentalDecorators": true  
  }  
}
```

<https://www.typescriptlang.org/docs/handbook/decorators.html>

Decorators

```
function LogClass(constructor: Function) {  
  console.log('=== Class registered ===');  
  console.log('Name:', constructor.name);  
  console.log('Constructor params count:', constructor.length);  
  console.log('Methods on prototype:', Object.keys(constructor.prototype));  
  console.log('Source code:', constructor.toString());  
}
```

@LogClass

```
class Example {  
  constructor(public id: number, public name: string) {}  
  
  greet() { return 'hi'; }  
}
```

```
new Example(123, 'example');
```

```
// === Class registered ===  
// Name: Example  
// Constructor params count: 2  
// Methods on prototype: []  
// Source code: class Example {  
//   constructor(id, name) {  
//     this.id = id;  
//     this.name = name;  
//   }  
//   greet() { return 'hi'; }  
// }
```

Decorators

```
function LogMethod(  
  target: any,  
  propertyKey: string,  
  descriptor: PropertyDescriptor,  
) {  
  const original = descriptor.value;  
  
  descriptor.value = function (...args: any[]) {  
    console.log(`Calling ${propertyKey}`, args);  
    const result = original.apply(this, args);  
    console.log(`Result:`, result);  
    return result;  
  };  
}
```

```
class Example {  
  @LogMethod  
  sum(a: number, b: number) {  
    return a + b;  
  }  
}
```

```
new Example().sum(2, 5);  
// Calling sum [ 2, 5 ]  
// Result: 7
```

Decorators

```
function LogMethod(  
  target: any,  
  propertyKey: string,  
  descriptor: PropertyDescriptor,    class Example { ... }  
) {  
  ...  
}
```

target === Example.prototype

- You are modifying the method on the prototype, not each instance.

propertyKey === 'sum'

- Method name

```
interface PropertyDescriptor {  
  configurable?: boolean;  
  enumerable?: boolean;  
  value?: any;  
  writable?: boolean;  
  get?(): any;  
  set?(v: any): void;  
}
```

- This is where the real power of decorators lives. See the next slide.

Decorators

```
interface PropertyDescriptor {  
    configurable?: boolean;  
    enumerable?: boolean;  
    value?: any;  
    writable?: boolean;  
    get?(): any;  
    set?(v: any): void;  
}
```



```
{  
    value: Function,    // actual method of the class  
  
    writable: true,     // determines whether the associated property's value can be changed  
  
    enumerable: false, // controls whether the property will be included when enumerating  
    // (listing) the object's properties. for...in loops, Object.keys(), Object.assign(),  
    // JSON.stringify(), The spread (...) operator for objects  
  
    configurable: true // can it be deleted/overridden  
}
```

Decorators

```
function LogMethod(  
  target: any,  
  propertyKey: string,  
  descriptor: PropertyDescriptor,  
) {  
  const original = descriptor.value;  
  
  descriptor.value = function (...args: any[]) {  
    console.log(`Calling ${propertyKey}`, args);  
  
    // apply is a built-in method of all JavaScript functions. It calls a function by  
    explicitly specifying:  
    // the context (this inside the function)  
    // the arguments (as an array)  
    const result = original.apply(this, args);  
  
    console.log(`Result:`, result);  
    return result;  
  };  
}
```

See another example in the code...

Decorators

```
function LogParameter(  
  target: Object,  
  propertyKey: string | symbol,  
  parameterIndex: number  
) {  
  console.log(  
    `Parameter decorator executed:  
    class: ${target.constructor.name}  
    method: ${String(propertyKey)}  
    parameter index: ${parameterIndex}`  
  );  
}  
  
class Example {  
  greet(@LogParameter name: string, age: number) {  
    console.log(`Hello ${name}, age ${age}`);  
  }  
}
```

Decorators

```
function Readonly(target: any, propertyKey: string) {  
  let value: any;
```

```
  Object.defineProperty(target, propertyKey, {  
    get() {  
      return value;  
    },  
    set(newValue: any) {  
      if (!!value) {  
        throw new Error(`Property "${propertyKey}" is readonly`);  
      }  
      value = newValue;  
    },  
    enumerable: true,  
    configurable: false,  
  });  
}
```

```
class User {  
  @Readonly  
  role!: string;  
  
  constructor() {  
    this.role = 'admin';  
  }  
}
```

```
const user = new User();  
user.role = 'user'; // Error: Property "role" is  
readonly
```

Decorator Execution Order (Important Concept)

1. Decorators do not run when an object is created.
They run once, when the class definition is evaluated by JavaScript.

2. After evaluation, decorators are applied in a strict order:

- Property decorators
- Parameter decorators
- Method decorators
- Class decorators

This happens bottom-up, starting from the smallest element and moving outward.

Decorator Factories

If we want to customize how a decorator is applied to a declaration, we can write a decorator factory. A **Decorator Factory** is simply **a function that returns the expression that will be called by the decorator at runtime**.

We can write a decorator factory in the following fashion:

```
function color(value: string) {  
  // this is the decorator factory, it sets up  
  // the returned decorator function  
  return function (target: any) {  
    // this is the decorator  
    // do something with 'target' and 'value'...  
  };  
}
```

See another example in the code...

Reflection

Problem:

TypeScript decorators operate without runtime type metadata, making it impossible to inspect parameter, property, or return types during execution without explicit reflection support.

Reflection is the ability of a program to inspect and store information about itself at runtime.

TypeScript doesn't support reflection out of the box in JavaScript.
→ That's why [reflect-metadata](#) is needed.

Reflection

```
npm i reflect-metadata --save
```

tsconfig.json:

```
{
  "compilerOptions": {
    "experimentalDecorators": true,
    "emitDecoratorMetadata": true
  }
}
```

`emitDecoratorMetadata` → TypeScript
adds type metadata automatically

app.ts:

```
import 'reflect-metadata';
```


Reflection

```
import 'reflect-metadata';
```

```
function LogType(target: any, propertyKey: string) {  
  const type = Reflect.getMetadata('design:type', target, propertyKey);  
  console.log(`${propertyKey} type:`, type.name);  
}
```

```
class User {  
  @LogType  
  age!: number;  
  
  @LogType  
  name!: string;  
}
```

```
// age type: Number  
// name type: String
```

Reflection

`design:type`

```
class User {  
  name: string;  
}
```

```
Reflect.getMetadata('design:type', User.prototype, 'name');  
// → String
```

Reflection

`design:paramtypes`

```
class UserService {}
```

```
class Controller {  
    constructor(service: UserService) {}  
}
```

```
Reflect.getMetadata('design:paramtypes', Controller);  
// → [UserService]
```

Reflection

`design:returntype`

```
class Example {  
  sum(): number {  
    return 1;  
  }  
}
```

```
Reflect.getMetadata('design:returntype', Example.prototype, 'sum');  
// → Number
```

Reflection

```
Reflect.defineMetadata(  
  metadataKey,  
  metadataValue,  
  target,  
  propertyKey?  
);
```

```
Reflect.hasMetadata(metadataKey, target, propertyKey?);
```

```
Reflect.deleteMetadata(metadataKey, target, propertyKey?);
```

```
Reflect.getMetadataKeys(target, propertyKey?);
```

<https://www.npmjs.com/package/reflect-metadata>

See another example in the code...

Stage 3 Decorators

Legacy decorators are powerful, but they rely on runtime reflection and implicit metadata.

TypeScript decorators were never part of JavaScript – they were an experimental extension.

New JavaScript decorators (Stage 3)

- Part of the ECMAScript proposal
- Supported in modern TS (`experimentalDecorators: false`)
- Different API

The JavaScript community wanted decorators that are:

- predictable
- standard
- explicit
- analyzable by tooling

```
{  
  "compilerOptions": {  
    "target": "ES2022",  
    "useDefineForClassFields": true  
  }  
}
```

Stage 3 Decorators

Mental model change

Legacy decorators:

- work with the **prototype**
- mutate the **descriptor**
- can modify the method directly
- run before the class is fully created

New decorators:

- do NOT touch the **prototype**
- do NOT receive a **descriptor**
- return a new function / initializer
- work through a **context** object

Stage 3 Decorators

`(value, context) => { ... }`

value:

class	→	constructor
method	→	function
field	→	undefined (initial value)
accessor	→	getter/setter <code>get age() { return 30 }</code>

context:

```
type Context = {  
  kind: "class" | "method" | "field" | "accessor";  
  name: string | symbol;  
  private?: boolean;  
  static?: boolean;
```

```
// addInitializer registers a function to run after the element is fully initialized.  
// - For class fields/methods: runs once per instance, after the field/method is set up.  
// - For classes: runs once after the class is fully defined, not per instance.
```

```
  addInitializer?(initializer: () => void): void;  
};
```


Stage 3 Decorators

```
interface ClassFieldDecoratorContext {  
    kind: "field";  
    name: PropertyKey;  
    static: boolean;  
    private: boolean;  
    addInitializer(initializer: (this: any) => void): void;  
}
```

```
interface ClassMethodDecoratorContext {  
    kind: "method";  
    name: PropertyKey;  
    static: boolean;  
    private: boolean;  
    addInitializer(initializer: (this: any) => void): void;  
}
```

Stage 3 Decorators

```
interface ClassAccessorDecoratorContext {  
    kind: "accessor";  
    name: PropertyKey;  
    static: boolean;  
    private: boolean;  
    addInitializer(initializer: (this: any) => void): void;  
}
```

```
interface ClassDecoratorContext {  
    kind: "class";  
    name: string;  
    addInitializer(initializer: (this: any) => void): void;  
}
```

Stage 3 Decorators

```
function LogClass<T extends new (...args: any[]) => any>
(value: T, context: ClassDecoratorContext) {
  console.log('Decorator executed');
  context.addInitializer(() => {
    console.log(`Class initializer for "${context.name}"`);
  });
  return class extends value {
    constructor(...args: any[]) {
      console.log(`Creating instance of ${context.name}`);
      super(...args);
    }
  };
}
```

```
@LogClass
class Example {
  constructor() {
    console.log('Inside constructor');
  }
}
new Example();
```

```
// Decorator executed
// Class initializer for "Example"
// Creating instance of Example           - when instance is created
// Inside constructor
```

Stage 3 Decorators

```
function LogMethod(value: Function, context: ClassMethodDecoratorContext) {  
  console.log('Decorator executed');  
  context.addInitializer(function () {  
    console.log(`LogMethod addInitializer`);  
  });  
  return function (this: any, ...args: any[]) {  
    console.log(`LogMethod return function`);  
    return value.apply(this, args);  
  };  
}
```

```
class Example {  
  constructor() {  
    console.log('Inside constructor');  
  }  
  @LogMethod  
  printSum(a: number, b: number) {  
    console.log('Sum is', a + b);  
  }  
}
```

```
const e = new Example();  
e.printSum(2, 3);
```

```
// Decorator executed  
// LogMethod addInitializer      - when instance is created  
// Inside constructor  
// LogMethod return function     - when the method is called  
// Sum is 5
```

Stage 3 Decorators

```
function LogField(value: any, context: ClassFieldDecoratorContext) {  
  console.log('Decorator executed');  
  context.addInitializer(function () {  
    console.log(`LogField addInitializer`);  
  });  
  return function (this: any, initialValue: any) {  
    console.log(`LogField return function`);  
    return initialValue;  
  };  
}
```

```
class Example {  
  @LogField x = 42;  
  constructor() {  
    console.log('Inside constructor');  
  }  
}  
const e = new Example();
```

```
// Decorator executed  
// LogField return function  
// LogField addInitializer  
// Inside constructor
```

- when instance is created

Stage 3 Decorators

```
function LogSetter(value: (v: any) => void, context: ClassSetterDecoratorContext) {  
    console.log(`Setter decorator executed for "${String(context.name)}"`);  
    context.addInitializer(function () {  
        console.log(`LogSetter addInitializer`);  
    });  
    return function (this: any, newValue: any) {  
        console.log(`Setting "${String(context.name)}" to`, newValue);  
        value.call(this, newValue);  
    };  
}
```

```
class User {  
    private _age = 0;  
  
    @LogSetter  
    set age(value: number) {  
        this._age = value;  
    }  
}
```

```
const user = new User();  
user.age = 30;
```

```
// Setter decorator executed for "age"  
// LogSetter addInitializer  
// Setting "age" to 30
```

- when instance is created
- when setter is called

Stage 3 Decorators

```
function LogSetterFactory(prefix: string) {  
  return function LogSetter(  
    value: (v: any) => void, context: ClassSetterDecoratorContext  
  ) {  
    console.log(`${prefix} decorator executed for "${String(context.name)}"`);  
  
    context.addInitializer(function () {  
      console.log(`${prefix} addInitializer`);  
    });  
  
    return function (this: any, newValue: any) {  
      console.log(`${prefix} setting "${String(context.name)}" to`, newValue);  
      value.call(this, newValue);  
    };  
  };  
}
```

```
class User {  
  private _age = 0;  
  
  @LogSetterFactory("MyPrefix")  
  set age(value: number) {  
    this._age = value;  
  }  
}
```

```
const user = new User();  
user.age = 30;
```

Stage 3 Decorators

Legacy Decorators

- Simple to use and widely supported.
- No access to type information at runtime (unless using [reflection metadata](#)).
- Cannot reliably execute code at instance creation without hacks.
- Limited control over fields, getters/setters, and methods after declaration.
- Used in major frameworks (NestJS, Angular, etc.) because of stability, compatibility, and mature ecosystem.
- DI containers, validation, and metadata in these frameworks rely on legacy decorators and [reflect-metadata](#).
- Transitioning to new decorators in these frameworks is currently impractical due to scale and tooling dependencies.

Stage 3 Decorators

- Part of the ECMAScript proposal
- Each decorator receives a structured [context](#) object describing the target.
- [addInitializer](#) allows safe execution of code when an instance is created.
- [Returning a function](#) allows modifying methods, accessors, and fields directly.



Дякуємо за увагу!